Chapitre 0 — Avant tout | Partie C — Déplacer une lettre | leçon 0.33 sur 153 | difficulté 1 / 10

0.33. Écrire soi-même sa fonction

mon_deplace_x, écrite à la main : le 0.17 rangé dans une fonction. C'est ce qu'il y a dans deplace_x.

1. Ce qu'il faut comprendre

Toutes ces fonctions sont écrites dans le même langage que tes programmes. Ici, on écrit la nôtre, `mon_deplace_x` : c'est, ligne pour ligne, ce que la console a dans `deplace_x`.

Ce qui est nouveau ici : écrire une fonction. `uint8_t mon_deplace_x(uint8_t colonne, uint8_t ligne, uint8_t tuile, uint8_t pas) { … }`. Le premier `uint8_t` dit ce que la fonction **rend** (un octet, la colonne) ; entre les parenthèses, ses **paramètres**, ce qu'elle reçoit ; entre les accolades, ce qu'elle fait.

Les paramètres sont des copies. Quand on appelle `mon_deplace_x(x, 0, ALPHABET[0], 5)`, `colonne` reçoit une copie de `x` (0), `pas` reçoit 5. La fonction change `colonne` et `pas` tant qu'elle veut : `x`, lui, ne bouge pas.

`return colonne;` rend la colonne d'arrivée, et termine la fonction. C'est cette valeur que `x = …` range dans `x`.

Ici, `x =` est obligatoire. Le rangement automatique du 0.23 ne vaut que pour les fonctions de la console. `mon_deplace_x(x, …);` seule ferait bouger la lettre, mais `x` resterait à 0 : c'est la règle normale du C.

`break` sort de la boucle `while` tout de suite : au bord de l'écran, il n'y a plus de pas possible.

> **Les fonctions de cette leçon — et le tuto de chacune**
>
> - `#include <poser>` → 0.1.3. La fonction poser() — poser une tuile
> - `#include <effacer>` → 0.16.3. La fonction effacer() — effacer des cases
> - `#include <ALPHABET>` → 0.1.4. ALPHABET — les lettres de la police

📖 Les mots de cette leçon — si tu débutes

- **accolades { }** — Elles entourent un bloc : les lignes qui vont ensemble (le corps d'une fonction, d'une boucle, d'un « if »). Chaque « { » a son « } ».
- **affectation (=)** — « x = 5; » : la boîte x prend la valeur 5. Ce n'est pas une égalité : c'est un rangement.
- **argument** — Une valeur donnée à une fonction quand on l'appelle, entre ses parenthèses : dans texte(5, 8, "SALUT"), 5, 8 et "SALUT" sont les arguments.
- **bloc** — Un groupe de lignes entre accolades { }, qui s'exécutent ensemble.
- **booléen (vrai / faux)** — Le résultat d'une condition : vrai (1) ou faux (0). « true » veut dire vrai, « false » faux.
- **boucle** — Des instructions qui se répètent : « while » (tant que…) ou « for » (pour un compteur qui va de… à…).
- **boucle du jeu** — « while (true) { image(); … } » : ce qui se répète 60 fois par seconde, sans fin, pendant tout le jeu.
- **break** — Sort tout de suite de la boucle (ou du « switch ») où l'on est.
- **commentaire (//)** — Tout ce qui suit « // » jusqu'au bout de la ligne : une phrase pour l'humain, que la console ignore.
- **condition** — Une question dont la réponse est vrai ou faux : « score > 10 », « bouton(A) ». On la teste avec « if » ou « while ».
- **coordonnées** — Deux nombres qui disent où : la colonne (vers la droite) puis la ligne (vers le bas), à partir de (0, 0) en haut à gauche.
- **else (sinon)** — Ce que l'on fait quand la condition du « if » est fausse.
- **fonction** — Un groupe d'instructions qui a un nom, qu'on lance en écrivant son nom et des parenthèses. Celles de la console (texte, poser…) sont toutes prêtes ; on peut écrire les siennes.
- **for** — Une boucle qui compte : « for (uint8_t i = 0; i < 5; i++) » fait 5 tours, i valant 0, 1, 2, 3 puis 4.
- **if (si)** — Teste une condition : si elle est vraie, on fait le bloc qui suit.
- **image()** — Attend l'image suivante. La console en montre 60 par seconde : c'est ce qui règle la vitesse d'un jeu.
- **#include** — En haut du programme : « #include <texte> » demande la fonction texte() à la console ; « #include "fichier.cpp" » verse un de tes fichiers à cet endroit.
- **incrémenter (++)** — « x++ » ou « x += 1 » : x augmente de 1. « x-- » : x diminue de 1.
- **indice** — Le numéro d'une case dans un tableau, entre crochets : cases[3] est la quatrième case (on compte à partir de 0).
- **instruction** — Un ordre donné à la console, terminé par un point-virgule : « texte(1, 1, "OK"); ».
- **main** — « int main() { … } » : là où le programme commence. La console exécute d'abord la première ligne de main.
- **octet** — Un nombre de 0 à 255 : la plus petite boîte de la mémoire. Une cartouche en a 32 768.
- **opérateurs de calcul** — + plus, - moins, * fois, / divisé par (sans virgule : 7 / 2 donne 3), % le reste de la division (7 % 2 donne 1).
- **opérateurs de comparaison** — == égal, != différent, < plus petit, > plus grand, <= plus petit ou égal, >= plus grand ou égal. Le résultat est vrai ou faux.
- **opérateurs logiques** — && « et » (les deux doivent être vrais), || « ou » (un seul suffit), ! « pas ».
- **paramètre** — Dans la définition d'une fonction, la boîte qui reçoit une valeur donnée à l'appel : dans « void f(uint8_t x) », x est un paramètre.
- **point-virgule (;)** — Il termine une instruction, comme le point termine une phrase.

- **poser()** — Met une tuile dans le décor, sur une case de la grille de 20 × 18 (32 × 32 en tout, avec la partie cachée).
- **return** — Termine la fonction et rend une valeur à celui qui l'a appelée.
- **uint8_t** — Le genre de nombre de cette console : un entier de 0 à 255 (un octet). Au-delà, il repart à 0.
- **variable** — Une boîte avec un nom, dans la mémoire, qui retient un nombre qu'on peut lire et changer.
- **while (tant que)** — Répète un bloc tant qu'une condition est vraie.

🔍 Le programme, ligne par ligne — chaque ligne dite en français

`#include <poser>  // pose une tuile sur une case du fond`	Demande à la console « poser » (ce que c'est : pose une tuile sur une case du fond). Sans cette ligne, le compilateur refuserait de l'employer. (Le commentaire le dit : « pose une tuile sur une case du fond ».)
`#include <effacer>  // efface des cases, ou tout le fond`	Demande à la console « effacer » (ce que c'est : efface des cases, ou tout le fond). Sans cette ligne, le compilateur refuserait de l'employer. (Le commentaire le dit : « efface des cases, ou tout le fond ».)
`#include <ALPHABET>  // les lettres de la police : ALPHABET[0] est le A`	Demande à la console « ALPHABET » (ce que c'est : les lettres de la police : ALPHABET[0] est le A). Sans cette ligne, le compilateur refuserait de l'employer. (Le commentaire le dit : « les lettres de la police : ALPHABET[0] est le A ».)
`uint8_t x = 0;  // la colonne de la lettre`	Une variable : une boîte appelée x, qui retient un nombre de 0 à 255, et qui commence à 0. (Le commentaire le dit : « la colonne de la lettre ».)
`// NOTRE fonction : c'est le 0.17, rangé sous un nom.`	Un commentaire : une phrase pour celui qui lit (« NOTRE fonction : c'est le 0.17, rangé sous un nom. »). La console l'ignore.
`//  uint8_t  mon_deplace_x(uint8_t colonne, uint8_t ligne, uint8_t tuile, uint8_t pas)`	Un commentaire : une phrase pour celui qui lit (« uint8_t mon_deplace_x(uint8_t colonne, uint8_t ligne, uint8_t tuile, uint8_t pas) »). La console l'ignore.
`//  |  |  |`	Un commentaire : une phrase pour celui qui lit (« | | | »). La console l'ignore.
`//  |  |  +-- les PARAMÈTRES : ce qu'elle reçoit. Des COPIES :`	Un commentaire : une phrase pour celui qui lit (« | | +-- les PARAMÈTRES : ce qu'elle reçoit. Des COPIES : »). La console l'ignore.
`//  |  |  changer colonne ne change pas x.`	Un commentaire : une phrase pour celui qui lit (« | | changer colonne ne change pas x. »). La console l'ignore.
`//  |  +---------- son nom`	Un commentaire : une phrase pour celui qui lit (« | +---------- son nom »). La console l'ignore.
`//  +---------- ce qu'elle REND : un octet (la colonne d'arrivée)`	Un commentaire : une phrase pour celui qui lit (« +---------- ce qu'elle REND : un octet (la colonne d'arrivée) »). La console l'ignore.
`uint8_t mon_deplace_x(uint8_t colonne, uint8_t ligne, uint8_t tuile, uint8_t pas) {`	Le début d'une fonction à nous, appelée mon_deplace_x : un groupe d'instructions qu'on lancera en écrivant mon_deplace_x(…). Elle reçoit des valeurs, rangées dans colonne, ligne, tuile, pas. Elle rend un nombre (avec return).
`uint8_t recule = pas > 127;  // -5 est rangé 251 : au-delà de 127, on recule`	Une variable : une boîte appelée recule, qui retient un nombre de 0 à 255, et qui commence à pas > 127. (Le commentaire le dit : « -5 est rangé 251 : au-delà de 127, on recule ».)

`if (recule) pas = 0 - pas;  // 251 redevient 5 : le nombre de cases`	Une décision : si recule alors on fait « pas = 0 - pas ». Sinon, on passe. (Le commentaire le dit : « 251 redevient 5 : le nombre de cases ».)
`poser(colonne, ligne, tuile);  // la lettre à sa place de départ`	Appel de la fonction poser de la console, avec les arguments colonne, ligne, tuile. Ce qu'elle fait : pose une tuile sur la grille du fond — un nom de Tuile, ou un numéro. Un 4e argument est une TAILLE : poser(2, 3, MUR, 5) dessine MUR cinq fois plus grand dans le fond (colonne et ligne en clair). (Le commentaire le dit : « la lettre à sa place de départ ».)
`while (pas > 0) {  // tant qu'il reste des pas`	Une boucle : tant que pas est plus grand que 0, on recommence ce qui est entre les accolades. (Le commentaire le dit : « tant qu'il reste des pas ».)
`for (uint8_t i = 0; i < 15; i++) image();  // un quart de seconde`	Une boucle qui compte : i part de 0 ; on continue tant que i est plus petit que 15 ; et après chaque tour, i augmente de 1. (Le commentaire le dit : « un quart de seconde ».)
`if (recule && colonne == 0) break;  // bord gauche : on sort de la boucle`	Une décision : si recule et colonne est égal à 0 alors on fait « break ». Sinon, on passe. (Le commentaire le dit : « bord gauche : on sort de la boucle ».)
`if (!recule && colonne == 19) break;  // bord droit  : on sort de la boucle`	Une décision : si pas recule et colonne est égal à 19 alors on fait « break ». Sinon, on passe. (Le commentaire le dit : « bord droit : on sort de la boucle ».)
`effacer(colonne, ligne, 1);  // 1. efface l'ancienne place`	Appel de la fonction effacer de la console, avec les arguments colonne, ligne, 1. Ce qu'elle fait : efface un texte à l'écran : « quoi » est le texte lui-même, ou un nombre de cases. Avec un Mot : effacer(SALUT). (Le commentaire le dit : « 1. efface l'ancienne place ».)
`if (recule) colonne--;  // 2. une colonne plus à gauche…`	Une décision : si recule alors on fait « colonne-- ». Sinon, on passe. (Le commentaire le dit : « 2. une colonne plus à gauche… ».)
`else colonne++;  //  …ou plus à droite`	Sinon (si la condition d'avant était fausse), on fait « colonne++ ». (Le commentaire le dit : « …ou plus à droite ».)
`poser(colonne, ligne, tuile);  // 3. la lettre à sa nouvelle place`	Appel de la fonction poser de la console, avec les arguments colonne, ligne, tuile. Ce qu'elle fait : pose une tuile sur la grille du fond — un nom de Tuile, ou un numéro. Un 4e argument est une TAILLE : poser(2, 3, MUR, 5) dessine MUR cinq fois plus grand dans le fond (colonne et ligne en clair). (Le commentaire le dit : « 3. la lettre à sa nouvelle place ».)
`pas--;  // un pas de moins à faire`	pas diminue de 1. (Le commentaire le dit : « un pas de moins à faire ».)
`}`	L'accolade fermante : la fin du bloc ouvert plus haut par « { ».
`return colonne;  // REND la colonne d'arrivée, et termine`	La fonction s'arrête ici et rend colonne à celui qui l'a appelée. (Le commentaire le dit : « REND la colonne d'arrivée, et termine ».)
`}`	L'accolade fermante : la fin du bloc ouvert plus haut par « { ».
`int main() {`	Le début du programme : la console commence ici, par la première ligne qui suit.
`// « x = » est OBLIGATOIRE ici : mon_deplace_x est à nous, pas à la console.`	Un commentaire : une phrase pour celui qui lit (« « x = » est OBLIGATOIRE ici : mon_deplace_x est à nous, pas à la console. »). La console l'ignore.

`// Elle reçoit une copie de x, et REND la colonne d'arrivée ; on la range.`	Un commentaire : une phrase pour celui qui lit (« Elle reçoit une copie de x, et REND la colonne d'arrivée ; on la range. »). La console l'ignore.
`x = mon_deplace_x(x, 0, ALPHABET[0], 5);  // x vaut 5`	x prend une nouvelle valeur : mon_deplace_x(x, 0, ALPHABET[0], 5). (Le commentaire le dit : « x vaut 5 ».)
`x = mon_deplace_x(x, 0, ALPHABET[0], -5);  // x vaut 0`	x prend une nouvelle valeur : mon_deplace_x(x, 0, ALPHABET[0], -5). (Le commentaire le dit : « x vaut 0 ».)
`while (true) {`	La boucle du jeu : tout ce qui est entre les accolades recommence sans fin, 60 fois par seconde avec image().
`image();`	Attendre l'image suivante : la console en montre 60 par seconde. C'est ce qui règle la vitesse du jeu.
`}`	L'accolade fermante : la fin du bloc ouvert plus haut par « { ».
`}`	L'accolade fermante : la fin du bloc ouvert plus haut par « { ».

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1077 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

```
Gravé dans la cartouche :
  démarrage         52 o  ← la console, à l'allumage  — la mise en route de la console
  mon_deplace_x()  156 o  ← main()  — ta fonction
  main()            47 o  ← démarrage  — ta fonction
  VBlank             8 o  ← la console, à chaque image  — compte les images, 60 fois par seconde
  AttendreImage     28 o  ← image()  — attend l'image suivante
  AttendreVBlank    16 o  ← démarrage  — attend que l'écran soit libre
  AttendreAcces     12 o  ← EffacerCases  — attend de pouvoir écrire une case
  AdresseCase       10 o  ← poser(), effacer()  — trouve l'adresse d'une case de l'écran
  EffacerCases      12 o  ← effacer()  — efface des cases
  CopierTuiles      18 o  ← démarrage  — copie les lettres et les dessins en mémoire vidéo
  EffacerCarte      14 o  ← démarrage  — vide les deux cartes de l'écran
  Tuiles           704 o  ← CopierTuiles  — la police entière
Lettres gravées : toutes (poser() reçoit un numéro calculé)
Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EcrireTexte, EcrireNombre, Diviser, Reste, Multiplier,
DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette,
DonneesTransfert, Notes
Fonctions du langage appelées :
  poser()         × 2    26 o sur place, dans mon_deplace_x()  + AdresseCase (10 o)
  effacer()       × 1    15 o sur place, dans mon_deplace_x()  + AdresseCase (10 o), EffacerCases (12 o)
  image()         × 2     6 o sur place, dans mon_deplace_x(), main()  + AttendreImage (28 o)
Tes fonctions appelées :
  mon_deplace_x() × 2    36 o sur place, dans main()
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
#include <poser>      // pose une tuile sur une case du fond
#include <effacer>    // efface des cases, ou tout le fond
#include <ALPHABET>   // les lettres de la police : ALPHABET[0] est le A
```

2. UNE VARIABLE — X

```
uint8_t x = 0;        // la colonne de la lettre

// NOTRE fonction : c'est le 0.17, rangé sous un nom.
//
//   uint8_t      mon_deplace_x(uint8_t colonne, uint8_t ligne, uint8_t tuile, uint8_t pas)
//   |            |             |
//   |            |             +-- les PARAMÈTRES : ce qu'elle reçoit. Des COPIES :
//   |            |                 changer colonne ne change pas x.
//   |            +---------------- son nom
//   +-------------------------------- ce qu'elle REND : un octet (la colonne d'arrivée)
```

3. UNE FONCTION — MON_DEPLACE_X()

```
uint8_t mon_deplace_x(uint8_t colonne, uint8_t ligne, uint8_t tuile, uint8_t pas) {
  uint8_t recule = pas > 127;          // -5 est rangé 251 : au-delà de 127, on recule
  if (recule) pas = 0 - pas;           // 251 redevient 5 : le nombre de cases
  poser(colonne, ligne, tuile);        // la lettre à sa place de départ
  while (pas > 0) {                    // tant qu'il reste des pas
    for (uint8_t i = 0; i < 15; i++) image();   // un quart de seconde
    if (recule && colonne == 0) break;          // bord gauche : on sort de la boucle
    if (!recule && colonne == 19) break;        // bord droit  : on sort de la boucle
    effacer(colonne, ligne, 1);        // 1. efface l'ancienne place
    if (recule) colonne--;             // 2. une colonne plus à gauche…
    else colonne++;                    //    …ou plus à droite
    poser(colonne, ligne, tuile);      // 3. la lettre à sa nouvelle place
    pas--;                             // un pas de moins à faire
  }
  return colonne;                      // REND la colonne d'arrivée, et termine
}
```

4. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

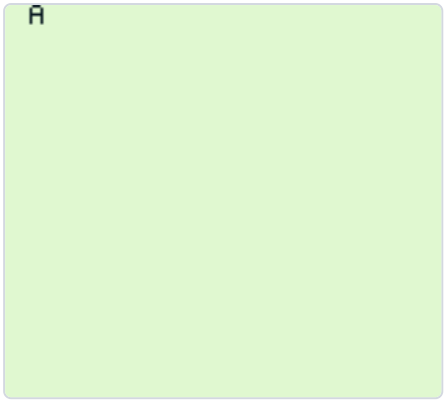
```
int main() {
  // « x = » est OBLIGATOIRE ici : mon_deplace_x est à nous, pas à la console.
  // Elle reçoit une copie de x, et REND la colonne d'arrivée ; on la range.
  x = mon_deplace_x(x, 0, ALPHABET[0], 5);    // x vaut 5
  x = mon_deplace_x(x, 0, ALPHABET[0], -5);   // x vaut 0

  while (true) {
    image();
  }
}
```

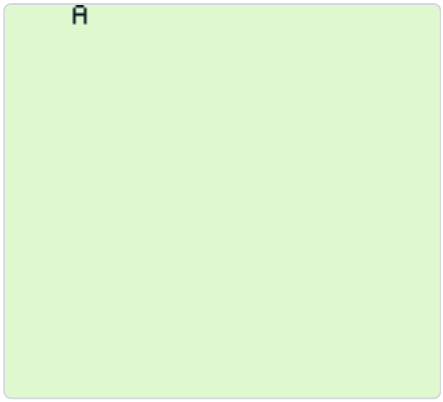
3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.

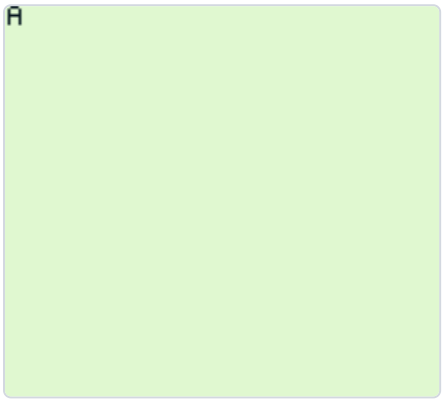
la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d’une seconde — image 60



au bout de trois secondes — image 180

4. Ce qu’on doit voir

> Exactement le 0.19 : un A qui avance de 5 cases, revient, et s’arrête. Mais cette fois, la fonction est écrite dans le programme.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d’être joués, et voici ce qu’ils ont mesuré.

- ✓ à l’aller, le A arrive en colonne 5 (« A »)
- ✓ au retour, le A revient en colonne 0 (« A »)
- ✓ x vaut la colonne rendue (x = 0)

Ensuite : **0.33.1. Écrire soi-même sa fonction — de base, ailleurs** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++